

```

java.util.Arrays

// 0 and 1 are not prime.
isPrime[0] = false;
isPrime[1] = false;

for (int current = 0; current <= MAX; current++)
{
    if (isPrime[current])
    {
        // This number is prime! Print it
        System.out.print(current + " ");

        // All multiples of this number
        int compositeNumber = current * 2;
        while (compositeNumber <= MAX)
        {
            isPrime[compositeNumber] = false;
            compositeNumber += current;
        }
    }
}

```

Compte Rendu projet POO

proposé par les étudiants :

Saidoun Djahid 201400008325

Ameur Mounira 161631102737

A-Introduction et énoncé

Ce projet java consiste en l'élaboration d'une mini application en java, cette application consistera en un logiciel de gestion de stock utilisant une interface graphique ou bien la console .

L'objectif de ce projet est la création d'un programme en langage java, dont la fonction principale est la gestion est la gestion d'un stock (regroupant un ensemble de produits).

Il s'agira en fait de créer un mini logiciel qui contient toutes les manipulations courantes (listing de produits actuellement présents dans le stock, procédure d'insertion d'un nouveau produit, la modification des informations d'un produit inscrit dans la «base» des produits...

Enonce :

Il a été demandé de :

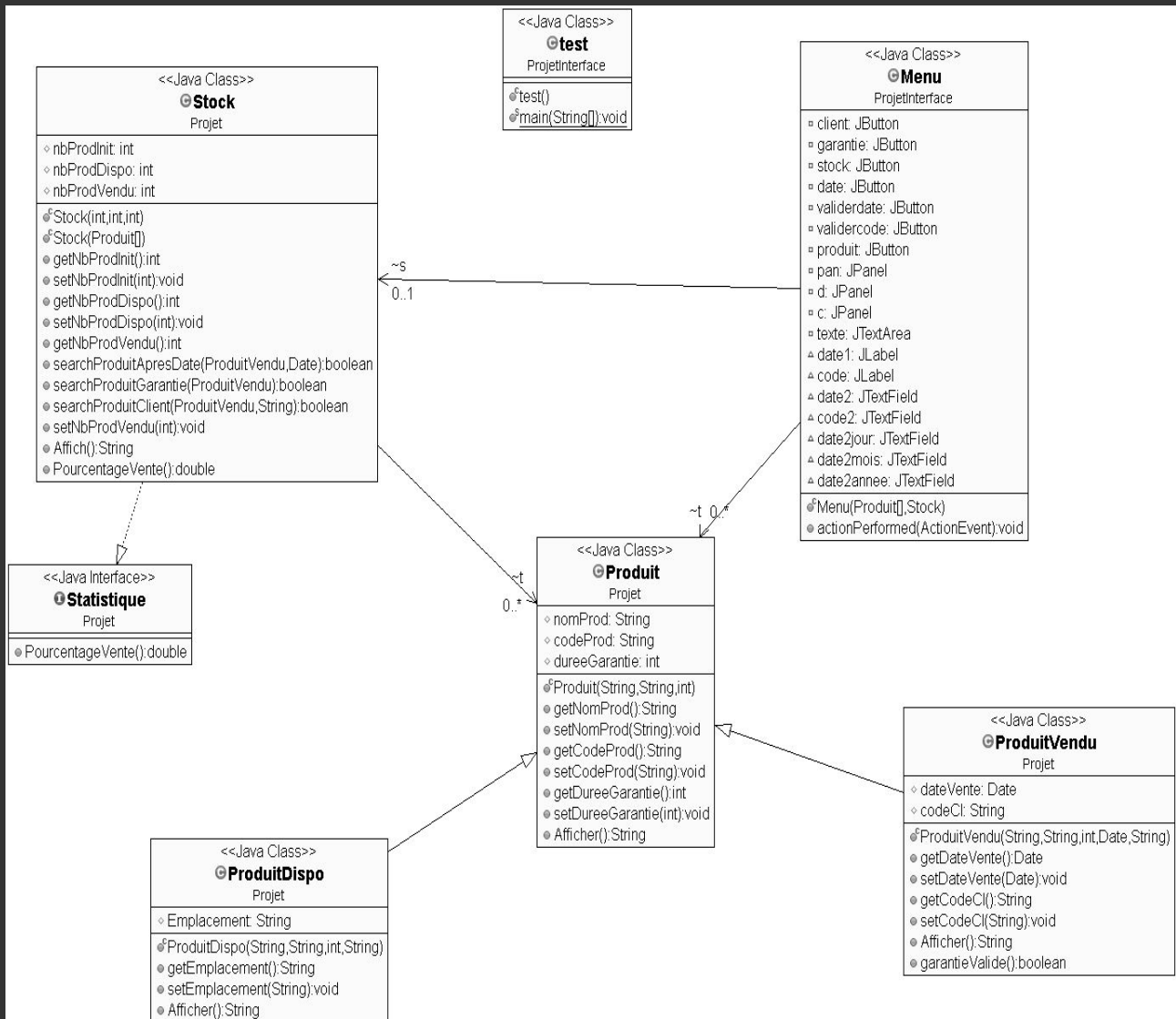
1- donner la modélisation du system

2- implémenter la méthode sous forme d'une application graphique en java, la classe principale doit avoir un menu avec les fonctionnalités suivantes : *

- renseigner les information d'un stock s
- initialiser un tableau de 30 éléments avec x produits vendus et y produits disponibles (tel que $x+y \leq 30$)
- afficher les produits vendus après une date donnée d
- afficher les produits qui sont encore couverts par la garantie.
- afficher les produits vendus pour 1 client C
- afficher le taux de vente du stock s

B-Implémentation :

1-Schéma général de la modélisation



Ce schéma a été généré grâce à l'outil *ObjectAid* qu'on a installé sur l'environnement eclipse(il n'existe pas par défaut,et il faut l'ajouter grace a *install new Software* qui se trouve dans le *help* de Eclipse) ,il a permit de générer automatiquement (Reverse Conversion) les diagrammes en UML a partir des classes(.class) se trouvant dans le src de notre projet.

2-Les classes et interfaces utilisées :

1- la classe Produit :

Description :

Nous pouvons considérer cette classe comme une classe générique,dont chacune des classes ProdDispo et ProdVendu vont hériter.

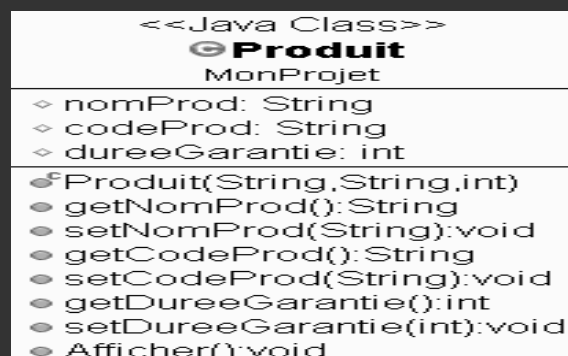
a)les attributs de la classe produit :

- nom du produit (String **nomProd**)
- le code du produit (String **codeProd**)
- durée de garantie en nombre de jours (exemple 365 jours de garantie) (int **dureeGarantie**)

b)les méthodes de la classe produit :

- lireProduit*
- afficheProduit*
- un constructeur (constructeur par défaut)*
- les setters() et les getters() :*

modélisation de cette classe en UML :



Les deux classes dérivées de la classe Produit qui héritent de ses attributs et ses méthodes :

2-La classe ProdDispo (les produits disponibles) :

Description :

La classe ProdDispo en plus des informations concernant un produit, elle est aussi définie par un emplacement de stockage (l'endroit où sont stockés les produits) **Emplacement** de type String.

-un constructeur par défaut ainsi que les méthodes *lireProduit* et *afficheProduit*. On aura aussi comme méthode une méthode dynamique qui calcule le nombre de produits disponibles **nbProdDispo**, qui s'appellera *ProdDispo()*.

-les setters() et getters().

UML :



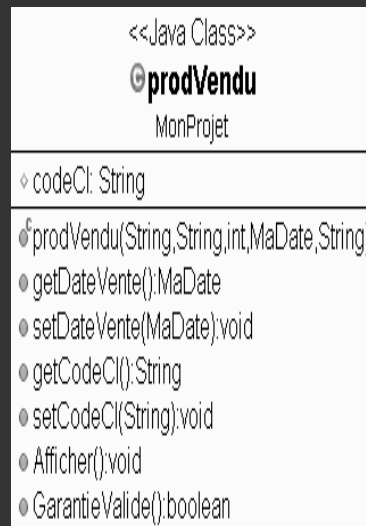
4-La classe ProdVendu (les produits disponibles) :

Description:

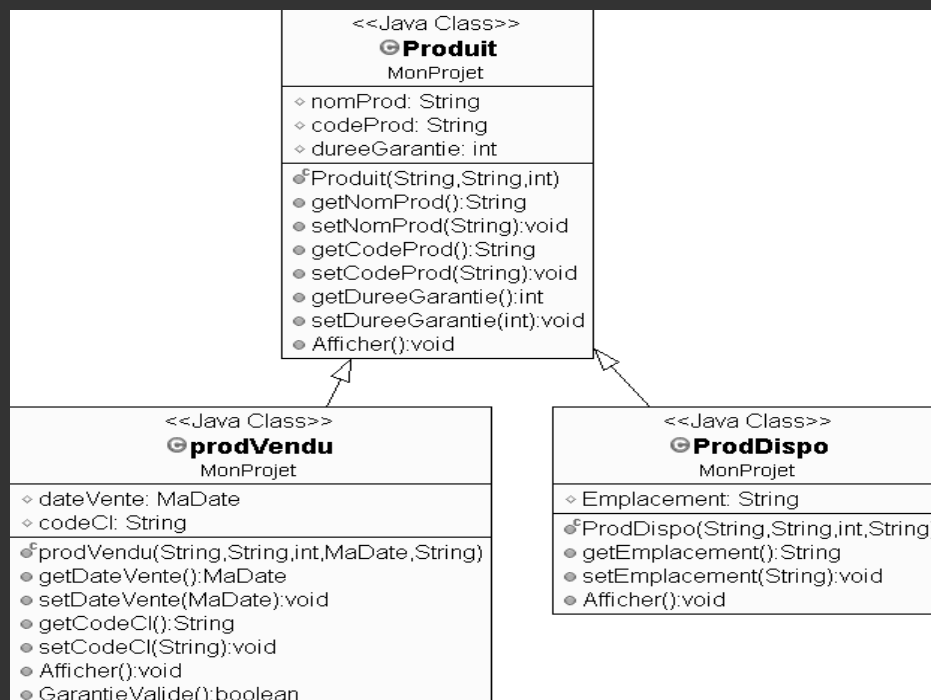
Un produit vendu est 1 produit qui a en plus ,une date de vente (Date **DateVente**),un code client (String **CodeCl**) et une méthode *GarantieValide()* qui vérifie si un produit vendu est couvert par la garantie ou non on voudrait aussi avoir la quantité des produits vendus.et un boolean qui indique si le produit est couvert par la garantie ou pas (boolean **garantie**). Et évidemment un constructeur par défaut ainsi que les méthodes *lireProduit* ,*afficheProduit*(héritées) et *GarantieValide()* qui permet d'affecter à

garantie true ou false selon que le produit est toujours couvert par la garantie ou non en calculant le nombre de jours entre la date actuelle et la date de vente et la méthode *Afficher()* qui affiche si le produit est couvert par la garantie ou pas.
 -setters() et getters().

UML :



Relation d'héritage entre Produit et ProduitDispo/ProduitVendu en UML :



4-La classe stock :

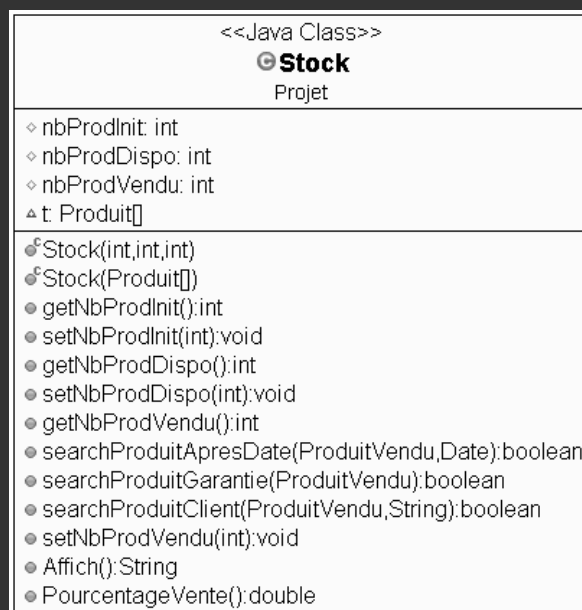
Description :

Un stock est défini par le nombre initial de produits (int **nbProdInit**) le nombre de produits disponibles (int **nbProdDispos**) et le nombre de produits vendus (int **nbProdVendus**). Quand un produit est vendu le stock est mis a jour.

les méthodes *afficheProduit()* et *lireProduit()* ainsi que les setters() et les getters().

La classe Stock implémente également l'interface statistique, elle implémente la méthode abstraite de cette interface (*pourcentageVente ()*) à travers une redéfinition(override).

UML :



6-L'interface statistique :

Description :

L'interface statistiques possède une méthode abstraite appelée *pourcentageVente ()* qui calcule le taux de vente d'un stock, ce taux sera calculé ainsi :

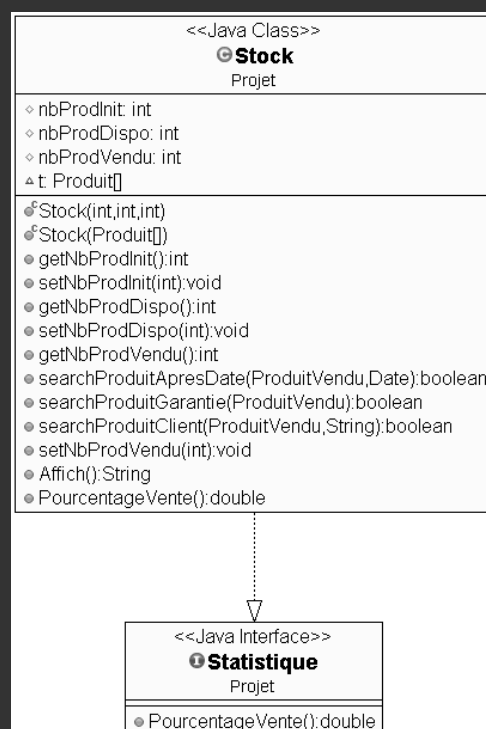
taux de vente = nombre de produits vendus/nombre initial de produits

UML :



lien entre statis- (implements) :

tiques et stock en uml

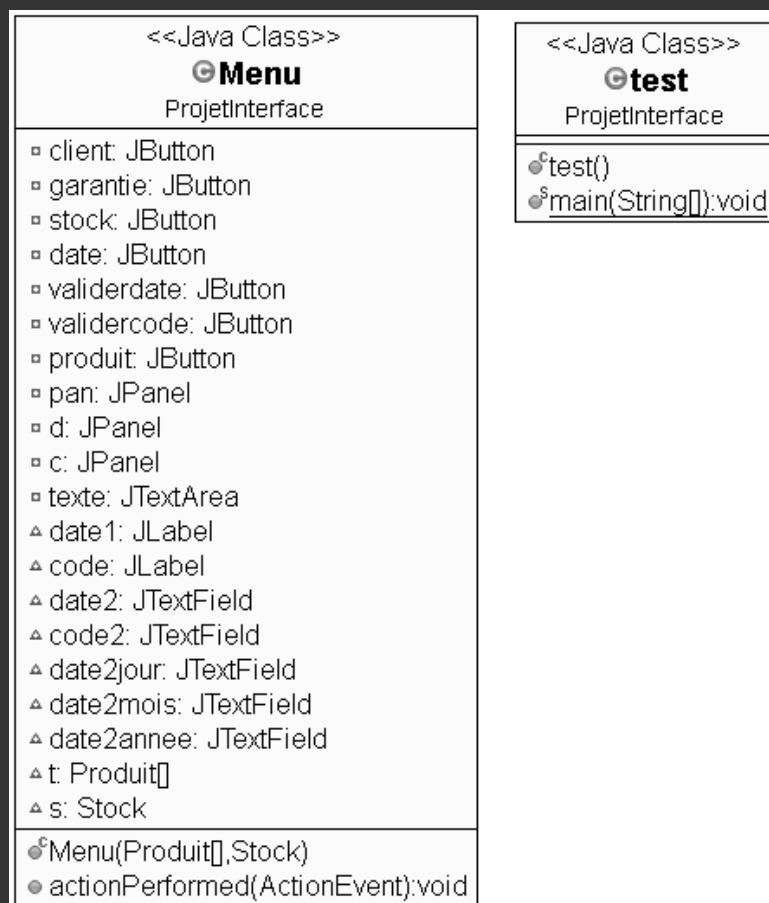


8-La classe Menu (l'interface graphique):

Description :

Evidemment le menu(le main) sous forme d'interface graphique (implémenté grâce a l'importation du fameux package swing et mise en place des buttons et de leurs événements OnClick(le click sur un bouton du menu prvoque l'appel a une méthode qui réalise un certain traitement X) ainsi que son static void main(qu'on a appelé test),qui vont permettre a l'utilisateur de l'application de déclarer et d'instancier des produits,d'y introduire des informations et des modifications sur ces produits,d'interroger la «base» (l'ensemble des produits instanciés et renseignés) à travers quelques requêtes,d'afficher des messages à travers l'interface ect ...

UML :



C-Conclusion :

Dans ce mini logiciel écrit en langage java,nous avons implémenté un programme capable de réaliser certaines tâches utiles dans la gestion d'un stock de produit.

La puissance du programme qu'on a implémenté réside dans sa portabilité,en effet il ne suffit que d'avoir le fichier .jar pour pouvoir exécuter le programme,au final nous avons un exécutable qui peut fonctionner sur toutes les machines(qui possèdent un JRE) indépendamment des OS et des SE utilisés,c'est un programme qui peut fonctionner partout,et qu'on peut facilement transporter(bon point pour la portabilité).

La rapidité de ce programme est aussi remarquable,chaque action que l'utilisateur veut effectuer se fait en 1 seconde maximum(toute action confondue) ce qui n'est pas négligeable sachant que ce genre de logiciel est souvent utilisé au niveau des administrations et RH(Gain de temps au niveau de la gestion).

